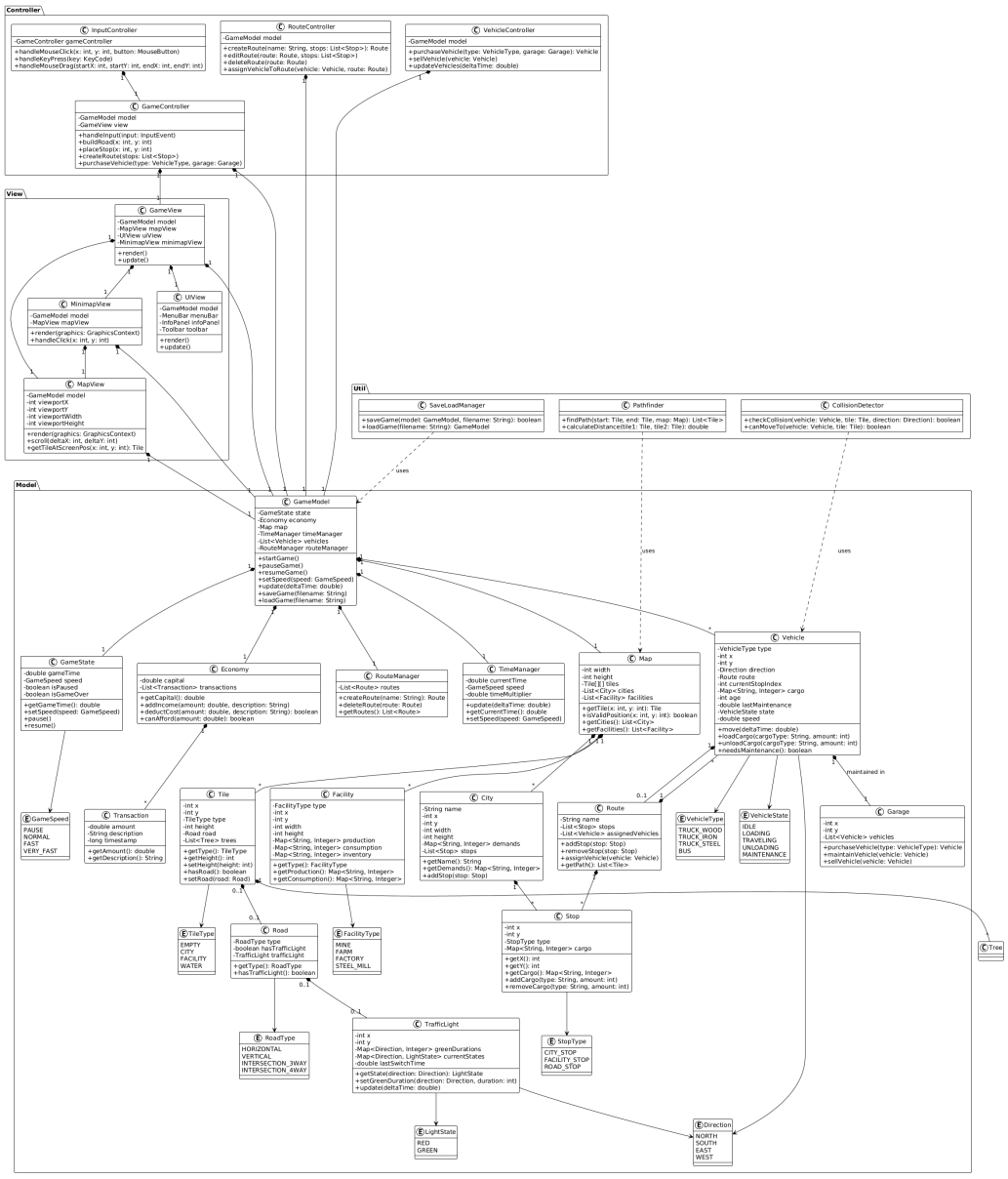
Class Diagram

This page contains the class diagram for Mini Transport Tycoon, showing the system architecture and relationships between classes.



Architecture Overview

The system follows a **Model-View-Controller (MVC)** architecture pattern with clear separation between:

- **Model**: Game logic, state management, business rules
- **View**: User interface, rendering, display
- **Controller**: User input handling, coordination between Model and View

Main Packages

Model Package

Contains the core game logic and data structures:

- **GameModel**: Main game controller managing all game systems
- **GameState**: Tracks game state (time, speed, pause status)
- **Map**: Manages the game map and tiles
- **Tile**: Represents individual map tiles
- **Road**: Represents road segments
- **City**: Represents cities on the map
- **IndustrialFacility**: Represents industrial facilities
- **Vehicle**: Base class for all vehicles
- **Route**: Represents vehicle routes
- **Stop**: Represents stops where vehicles load/unload
- **Economy**: Manages economic calculations
- **TimeManager**: Handles game time progression

View Package

Contains UI components and rendering:

- **GameView**: Main game view component
- **MapRenderer**: Renders the game map
- **UIManager**: Manages UI components
- **MinimapView**: Displays and handles minimap

Controller Package

Contains input handling and coordination:

- **GameController**: Main controller coordinating model and view
- **InputHandler**: Handles user input
- **ToolManager**: Manages tool selection and actions

Key Relationships

- **GameModel** aggregates **Map**, **Economy**, **TimeManager**, and manages collections of **Vehicle** and **Route**
- **Map** contains a grid of **Tile** objects
- **Tile** can contain **Road**, **Stop**, or be part of **City** or **IndustrialFacility**
- **Route** contains multiple **Stop** objects
- **Vehicle** is assigned to a **Route** and moves between **Stop** objects
- **GameController** coordinates between **GameModel** and **GameView**

Design Patterns Used

- **MVC Pattern**: Separation of concerns between Model, View, and Controller
- **Observer Pattern**: View observes Model changes for updates
- **Factory Pattern**: For creating vehicles and other game objects
- **Strategy Pattern**: For different vehicle behaviors and movement strategies

Comments